

Trabajo Practico 2

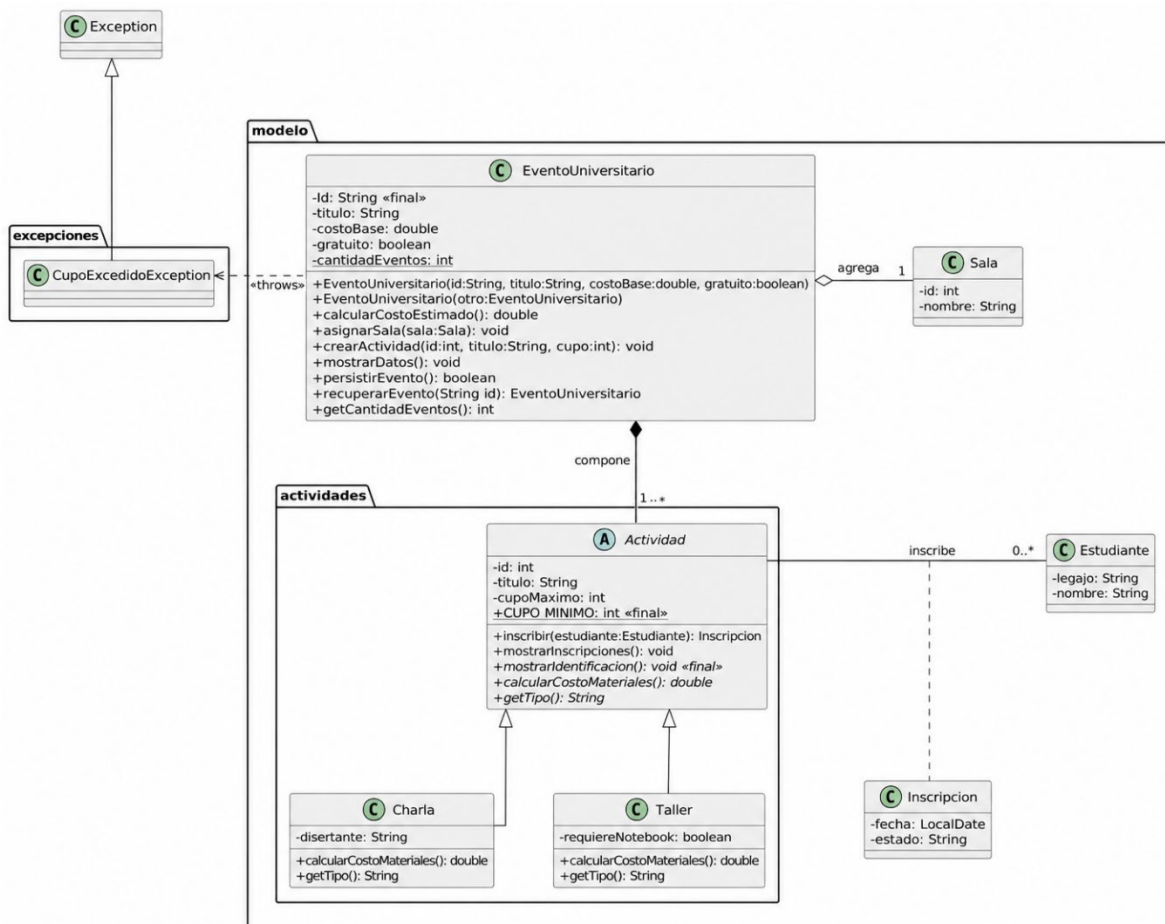
Programación Orientada a Objetos en Java

Unidad 2 - Organización, reutilización y recursos avanzados en POO

Ejercicio 1

Contexto: El sistema de eventos ha agregado varias clases que cumplen distintas funciones en el modelo y mantienen distinto grado de vínculos con otras clases, por lo que es conveniente modularizar el código para mejorar los niveles de encapsulamiento. Además, si bien el sistema funciona, debe poder responder correctamente ante fallos: cupos agotados, inscripciones que no alcanzan el cupo mínimo, datos inválidos y problemas al guardar o recuperar eventos desde archivo.

Se requiere: escalar el desarrollo e implementar correctamente en Java el siguiente modelo OO, incorporando correctamente el manejo de packages propuesto, manejo de excepciones para gestionar tolerancia a fallos y persistencia de objetos mediante serialización/deserialización.



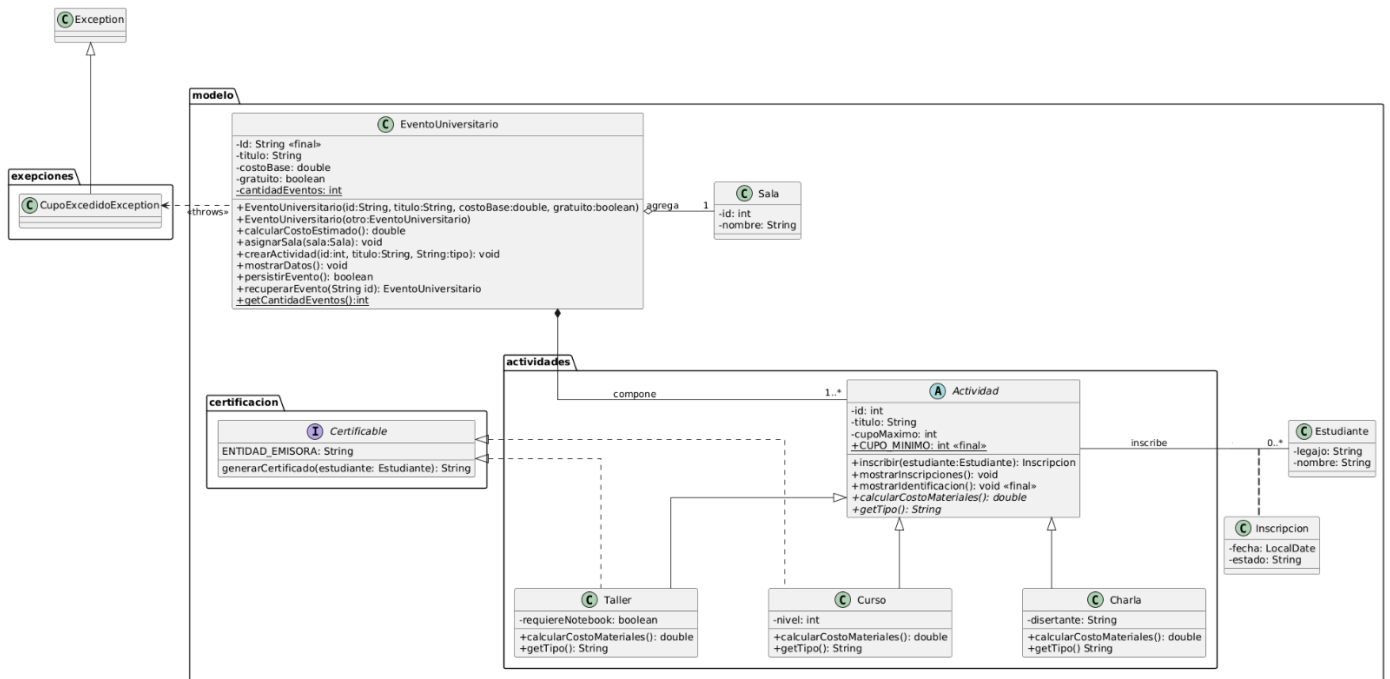
Resultado esperado: Código Java funcional ejecutable desde la clase App donde:

- Se modifique el método inscribir para lanzar CupoExcedidoException con throw y declarar throws. Atrapar y manejar adecuadamente esa excepción chequeada en la clase principal App.
- Persistir, mediante serialización de objetos, el evento creado.
- Manejar adecuadamente, de forma granular y completa las excepciones vinculadas a la persistencia de objetos.
- Implementar en App un flujo con try-catch-finally que intente inscribir, persistir y leer un evento.
- Mostrar por consola mensajes claros para cada catch.
- El programa debe mostrar al menos un caso exitoso y un caso fallido controlado por excepciones, sin mezclar validaciones complejas con la lógica principal.

Ejercicio 2

Contexto: Se necesita incorporar al sistema la posibilidad de emitir Certificados a los estudiantes que participan de diversas actividades en el marco de cada evento. Algunos tipos de actividades son certificables y otras no. En particular, se decide emitir certificados para talleres o cursos, pero no para charlas.

Se requiere: escalar el desarrollo e implementar correctamente en Java el siguiente modelo OO, incorporando correctamente el manejo de interfaces.



Resultado esperado: Código Java funcional ejecutable desde la clase App donde:

- Se creen estudiantes.
- Se creen eventos.
- Se asigne una sala a cada evento creado.
- Se construyan actividades para cada evento creado, incorporando un nuevo tipo de actividad posible Curso.
- Se inscriban alumnos en las actividades de cada evento.
- Se emitan certificados de asistencia para los alumnos que han participado de talleres y/o cursos. Las charlas no son certificables.
- Se muestren los certificados emitidos.
- Se muestren los datos de los eventos creados.

Ejercicio 3

Contexto: Se necesita incorporar al sistema dos nuevas funcionalidades:

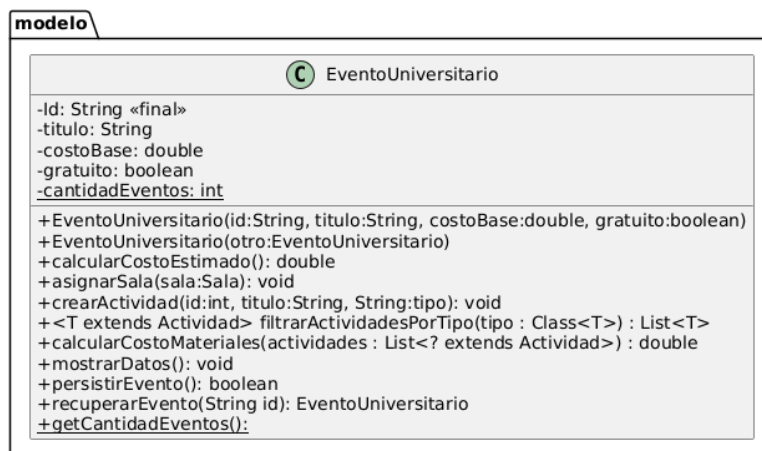
- La posibilidad de filtrar la lista de actividades de un evento por tipo de actividad: Charla, Taller o Curso.
- La posibilidad de calcular el costo de materiales de una lista de actividades formada por objetos del tipo Actividad o por objetos de cualquier subtipo de Actividad.

Se requiere: Agregar a la clase EventoUniversitario los métodos necesarios para resolver estas funcionalidades utilizando métodos parametrizados acotados y wildcards en Java. En particular, se deberán implementar y utilizar los siguientes métodos:

```
public <T extends Actividad> List<T> filtrarActividadesPorTipo(Class<T> tipo)

public double calcularCostoMateriales(List<? extends Actividad> actividades)
```

Estos métodos deben operar sobre las actividades que pertenecen al evento, respetando el modelo ya implementado, donde las actividades existen en el contexto del EventoUniversitario.



Resultado esperado: Código Java funcional ejecutable desde la clase App donde:

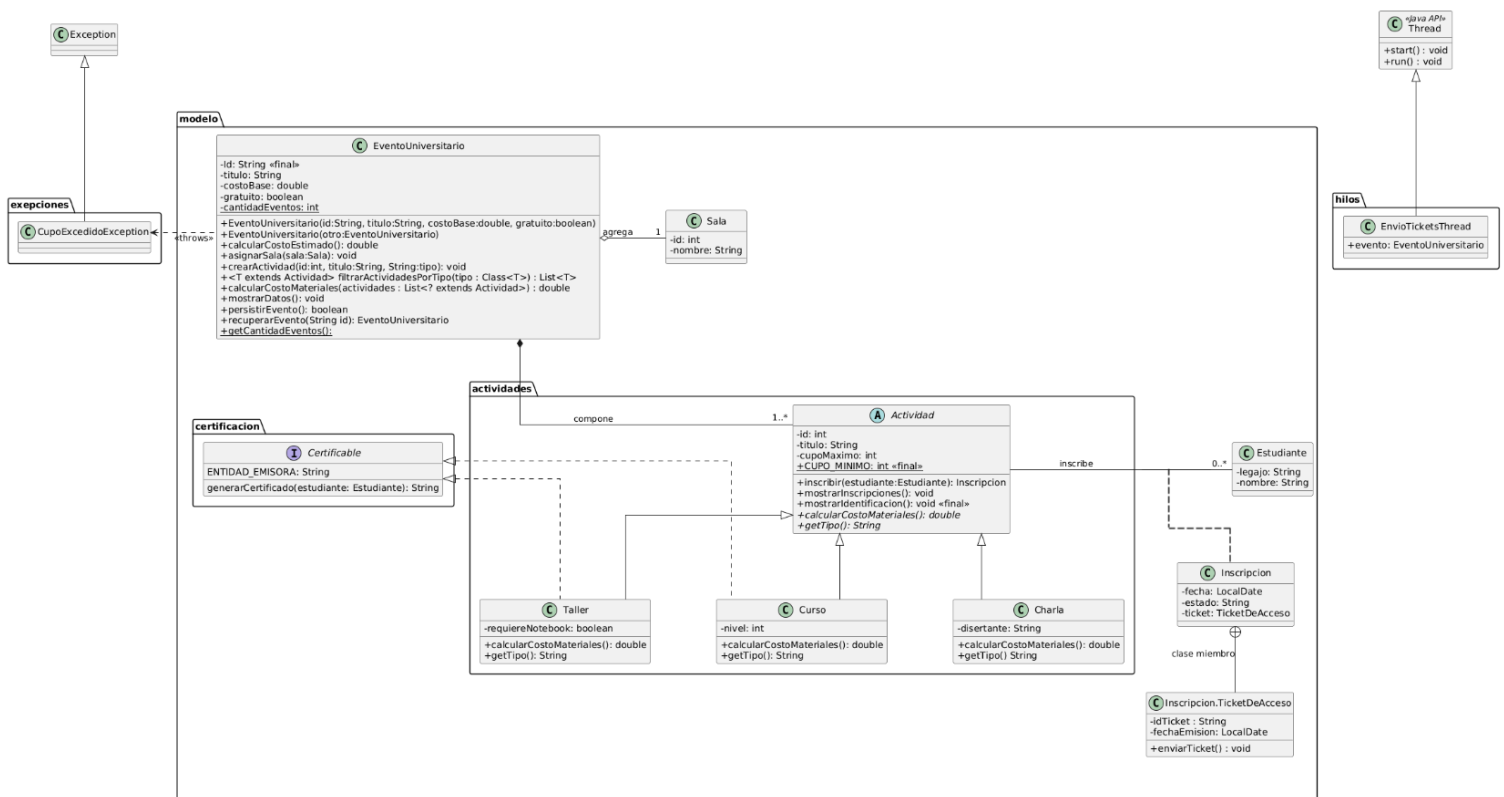
- Se creen estudiantes.
- Se creen eventos con su sala y sus actividades.
- Se inscriban los estudiantes en distintas actividades del evento.
- Se filtre la lista de actividades de cada evento por tipo concreto: Charla, Taller y Curso.
- Se muestre por consola la cantidad de actividades de cada tipo creadas para cada evento.
- Se calcule y muestre el costo de materiales correspondiente a cada tipo de actividad en un evento.
- Se evidencie que el filtrado devuelve listas correctamente tipadas, por ejemplo `List<Charla>`, `List<Taller>` y `List<Curso>`.

Ejercicio 4

Contexto: El sistema necesita incorporar la posibilidad de emitir tickets de acceso para las inscripciones confirmadas de los estudiantes en las actividades de un evento universitario.

Además, se requiere disponer de un proceso que permita enviar los tickets de acceso a los estudiantes inscriptos, sin interrumpir el funcionamiento general del sistema. Esto significa que el envío de tickets debe ejecutarse de forma concurrente, permitiendo que, mientras se realiza dicho envío, el programa principal pueda continuar mostrando información del evento, sus actividades e inscripciones.

Se requiere: escalar el desarrollo e implementar correctamente en Java el siguiente modelo OO, incorporando correctamente el uso de clases anidadas e hilos.



La clase TicketDeAcceso debe modelarse como una clase anidada miembro dentro de la clase Inscripcion, porque la existencia de un ticket solo tiene sentido en el contexto de una inscripción concreta. Un ticket debe emitirse únicamente cuando la inscripción se encuentra confirmada.

La clase EnvioTicketsThread, ubicada en el paquete hilos, debe implementarse como una clase pública e independiente encargada de ejecutar, en un hilo separado, el proceso de envío de todos los tickets generados para las inscripciones confirmadas de un evento.

Resultado esperado: Código Java funcional ejecutable desde la clase App donde:

- a. Se creen estudiantes.
- b. Se creen eventos universitarios cada uno con su sala y sus actividades.
- c. Se inscriba a los estudiantes en distintas actividades de cada evento.
- d. Se confirmen algunas de las inscripciones de estudiantes en las actividades de cada evento.
- e. Se genere un ticket de acceso por cada inscripción confirmada.
- f. Se inicie un proceso concurrente para enviar todos los tickets generados.
- g. Mientras el hilo de envío de tickets se ejecuta, el hilo principal debe continuar mostrando por consola los datos del evento, sus actividades y los estudiantes inscriptos.
- h. Debe evidenciarse en consola que existen dos flujos de ejecución: el hilo principal del programa y el hilo encargado del envío de tickets.

PAUTAS DE ENTREGA DEL TP

Para que la tarea se considere entregada debe:

1. Crear un repositorio en GitHub, de acceso público, cuyo nombre sea: PP_TP2_legajo
Por ejemplo: https://github.com/jperez/PP_TP1_50268
2. El repositorio debe contener lo siguiente:
 - 2.1. El proyecto de código desarrollado, completo hasta el ejercicio 4, generado desde INTELLIJ IDEA listo para clonar y probar.
 - 2.2. Un Readme con documentación sobre el proyecto implementado.
 - 2.3. Una captura de la salida por consola de una ejecución del programa.
3. Consignar en la entrega, la URL para clonar el repositorio vía https. (En el enlace Entregar Trabajo Práctico N° 2).

ATENCIÓN tener en cuenta:

- lo indicado anteriormente es requisito para que la tarea sea evaluada (VERIFICAR que se cumple con todos los requisitos ANTES de concretar la entrega, ya que en caso de que no se pueda clonar el repositorio, falte alguno de los archivos o no coincida en formatos o nombres con lo solicitado, no se califica y NO se otorga tiempo adicional para completar la entrega.
- COMPROBAR que el repositorio se puede clonar correctamente sin requerir permisos.
- SI SE COMPRUEBAN MODIFICACIONES EN EL REPO POSTERIORES A LA FECHA DE CIERRE DE LA ENTREGA EL TP SERÁ CONSIDERADO NO ENTREGADO.